Guía de operación del robot SCARA

ROS 2 Jazzy + micro-ROS en Ubuntu | Reporte No. 3 - Operación punto a punto

Juan David Botero Cubides - Mecatrónica, Universidad EIA

1. Cómo está armado el sistema

El control de bajo nivel (lazo de 1 kHz, PID, cinemática inversa y seguridad) corre **dentro del ESP32**, en C. ROS 2 solo envía consignas y recibe el estado. El ESP32 se conecta al PC por el cable USB: el **agente micro-ROS** lo convierte en un nodo más de ROS 2, y asi Python, RViz y los comandos `ros2` ven los tópicos del robot.

2. Antes de empezar (lista de chequeo)

- [ ] ESP32 conectado por USB y placa de potencia alimentada.
- [ ] Ningún monitor serie abierto (pio device monitor, Arduino, etc.): el puerto es de uno solo.
- [ ] Robot despejado de cables y objetos, y el corte de alimentación al alcance de la mano.
- [ ] Puerto detectado: ejecuta `ls /dev/ttyUSB*` y debe aparecer /dev/ttyUSB0.
- [ ] Firmware en modo micro-ROS ya cargado (se hace una sola vez; solo se repite si cambias algo del firmware).
- [ ] Archivos en `~/Trabajos/Robotica/Robot/scara_fw_microros/scara_fw/ros2/`: scara_client.py, scara_teleop_ros.py, scara_rviz.py, scara_rviz.launch.py, scara.urdf, scara.rviz.

3. Paso a paso para iniciar

Abre las terminales en este orden. Las terminales nuevas ya cargan ROS 2 porque tu `~/.bashrc` hace el `source`. Deja abiertas las terminales 1 y 2 durante toda la demostración.

Terminal 1 - El agente micro-ROS (puente con el ESP32)

```
docker run -it --rm -v /dev:/dev --privileged --net=host \
    microros/micro-ros-agent:jazzy serial --dev /dev/ttyUSB0 -b 115200 -v6
```

Después pulsa el botón **EN** del ESP32 para reiniciarlo. **Qué imprime:** mientras el robot se conecta, líneas de información del agente; ya conectado, un flujo continuo de líneas `debug` como las de abajo (es el tráfico del robot, es normal que no pare). Si eso aparece, la comunicación está viva.

```
[...] debug | SerialAgentLinux.cpp | recv_message | [==>> SER <<==] | client_key: 0x2AECB338 ...
[...] debug | DataWriter.cpp       | write        | [** <<DDS>> **]  | client_key: 0x00000001 ...
```

Terminal 2 - RViz y el puente de visualización

```
cd ~/Trabajos/Robotica/Robot/scara_fw_microros/scara_fw
ros2 launch ros2/scara_rviz.launch.py
```

Qué imprime: avisos de lanzamiento de tres procesos y se abre la ventana de RViz (vista desde arriba, con los dos círculos verdes del espacio de trabajo y el robot).

```
[INFO] [launch]: All log files can be found below ...
[INFO] [robot_state_publisher-1]: process started with pid [...]
[INFO] [python3-2]: process started with pid [...]
[INFO] [rviz2-3]: process started with pid [...]
```

Terminal 3 - Comprobar que todo está conectado

```
cd ~/Trabajos/Robotica/Robot/scara_fw_microros/scara_fw
ros2 node list
ros2 topic list
ros2 topic echo /scara/state --once
```

Qué imprime:

```
/robot_state_publisher
/rviz
/scara/scara_esp32      <- el robot (nodo que corre en el ESP32)
/scara_rviz

/joint_states   /scara/cmd   /scara/ee_position   /scara/event
/scara/goto     /scara/jog   /scara/markers       /scara/state

data:  -0.0, 0.0, 190.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...   (13 numeros; ver la tabla de la seccion 5)
```

Antes del Home el estado es q1 = 0, q2 = 0 y x = 190: no significa nada físico todavía, porque los encoders arrancan en cero al encender. **Si /scara/scara_esp32 no aparece, el agente no está conectado** (ver la sección 7).

Terminal 4 - Operar el robot (demostración)

Esta terminal es la que usas durante la demostración. **El Home es obligatorio después de cada encendido** (sin Home el firmware rechaza los movimientos).

```
cd ~/Trabajos/Robotica/Robot/scara_fw_microros/scara_fw
python3 ros2/scara_client.py home               # 1) referencia con los finales de carrera
python3 ros2/scara_client.py goto 120 80 0      # 2) consigna (x, y, z) en mm
python3 ros2/scara_client.py goto 100 -60 0
python3 ros2/scara_client.py goto 400 0         # 3) inalcanzable: demuestra la validacion
python3 ros2/scara_client.py ang 20 90          # 4) consigna articular en grados
python3 ros2/scara_teleop_ros.py                # 5) panel de control + manual con dead-man
python3 ros2/scara_client.py stop               # detener (mantiene la posicion)
python3 ros2/scara_client.py off                # apagar motores
```

4. Qué imprime cada comando (salidas esperadas)

home

```
#OK: home solicitado
#HOME: iniciando. Eje 2: primero un tope, luego el otro, y vuelve a donde estaba. (cualquier tecla =
    detener)
#HOME: eje 2 referenciado en el tope MIN (q = -143.00 deg). Ahora el tope contrario...
#HOME: eje 2 recorrido medido: de -143.00 a 132.76 deg (total 275.76 deg). Limites por software: -140.02
    a 129.78 deg.
#HOME: ahora el eje 1, buscando su primer tope...
#HOME: eje 1 referenciado en el tope MIN (q = -108.00 deg). Ahora el tope contrario...
#HOME: eje 1 recorrido medido: de -108.00 a 103.90 deg (total 211.90 deg). Limites por software: -105.02
    a 100.92 deg.
#HOME: terminado. Volviendo a la posicion inicial (q1 = -1.12, q2 = 0.00 deg)...
#LLEGO: q1=-1.121 q2=0.000 deg | medido x=189.96 y=-3.72 mm
```

El robot toca cada tope del eje 2 y luego del eje 1, mide su recorrido real y regresa a donde estaba. Los recorridos medidos pueden variar un poco en cada Home.

goto 120 80 0

```
#INFO: cinematica inversa (120.0, 80.0, 0.0) -> q1 = 4.78 deg, q2 = 84.88 deg (codo +1). Moviendo...
#LLEGO: objetivo x=120.00 y=80.00 z=0.00 | medido x=120.15 y=79.73 | error ex=0.153 ey=-0.271 mm |
    q1=4.651 q2=84.900 deg
```

La cinemática inversa se calcula en el ESP32. El *error* de #LLEGO sale de los encoders (posición medida contra la pedida); no es una medición externa con regla.

goto 400 0 (fuera del espacio de trabajo)

```
#ERROR: no se puede ir a (400.0, 0.0): inalcanzable (fuera del espacio de trabajo). Alcance: 50.0 a
    190.0 mm.
```

Otros errores de validación: z distinta de la altura fija (#ERROR: z = ... no es valida), punto singular, o que viole los límites articulares.

ang 20 90, stop, off, clear, status, gain

```
> ang 20 90   ->  #INFO: moviendo a q1 = 20.00 deg, q2 = 90.00 deg...   luego   #LLEGO: q1=... q2=... |
    medido x=... y=...
> stop        ->  #OK: detenido (mantiene posicion)
> off         ->  #OK: motores apagados
> clear       ->  #OK: falla borrada     (si habia falla: los ejes quedan SIN referencia, ejecuta home
    de nuevo)
> status      ->  #INFO: [HOLD] q1=4.70 q2=84.90 deg x=120.0 y=80.0 mm home J1:1 J2:1 falla:ninguna
> gain        ->  #INFO: J1 kp=5000 ki=3000 kd=180 kff=80 fric=70 | J2 kp=5000 ki=3000 kd=180 kff=80
    fric=70
```

5. Qué hace cada nodo

Nodo	Dónde corre	Qué hace	Qué muestra / publica
/scara/scara_esp32	En el ESP32	Recibe `/scara/goto`, `/scara/jog` y `/scara/cmd`; calcula la cinemática inversa, controla los motores en lazo cerrado con los encoders y vigila los finales de carrera y el dead-man. Publica el estado, la posición del efector y los eventos.	/scara/state, /scara/ee_position, /scara/event
micro_ros_agent	PC (Docker)	No es un nodo de lógica: es el puente. Traduce lo que el ESP32 envia por el cable USB a tópicos normales de ROS 2, y al reves.	Líneas debug (tráfico)
scara_client	PC (Python)	Se crea con cada comando de la terminal: publica la consigna o el comando y espera el evento final (#LLEGO, #HOME terminado o un error).	Los eventos que recibe
scara_teleop	PC (Python)	Panel de control: teclado con dead-man (espacio + flechas a 25 Hz), botones, mover en x,y,z o en grados, ganancias y línea de comandos.	Registro de eventos en la ventana
scara_rviz	PC (Python)	Convierte /scara/state en /joint_states y dibuja los círculos del espacio de trabajo, la consigna (esfera verde) y la estela del efector.	Nada en la terminal
robot_state_publisher	PC (ROS 2)	Con el modelo scara.urdf y los angulos de /joint_states calcula la posición de cada eslabón (transformadas TF).	Nada
rviz2	PC (ROS 2)	La ventana de visualización del robot.	La ventana

Tópicos

Tópico	Tipo	Sentido	Contenido
`/scara/goto`	geometry_msgs/Point	PC -> ESP32	Consigna x, y, z en mm. La z debe ser la altura fija.
`/scara/jog`	geometry_msgs/Vector3	PC -> ESP32	Manual a 25 Hz: x = dead-man (1 = pulsado), y = dirección J1, z = dirección J2 (-1, 0, 1).
`/scara/cmd`	std_msgs/String	PC -> ESP32	Comandos de texto: home, stop, off, clear, status, help, goto, ang, elbow, pulse, gain.
`/scara/state`	Float32MultiArray	ESP32 -> PC	13 números a 20 Hz (tabla de abajo).
`/scara/ee_position`	geometry_msgs/Point	ESP32 -> PC	Posición medida del efector en mm.
`/scara/event`	std_msgs/String	ESP32 -> PC	Mensajes: #OK, #INFO, #HOME, #LLEGO, #ERROR, #FALLA.
`/joint_states`, `/scara/markers`	JointState / MarkerArray	scara_rviz -> RViz	Ángulos del robot y dibujos auxiliares para RViz.

Los 13 datos de /scara/state

Posición	Significado
0, 1	q1, q2 en grados
2, 3	x, y del efector en mm
4	Modo: 0 OFF, 1 HOMING, 2 HOLD, 3 MOVE, 4 JOG, 5 PULSE, 6 FAULT
5	Falla: 0 ninguna, 1 error de seguimiento, 2 motor trabado, 3 final de carrera inesperado, 4 tiempo de Home agotado, 5 MIN y MAX del mismo eje a la vez
6, 7	Home hecho en J1, J2 (1 = si)
8	Finales de carrera presionados, en bits: 1 = J1 min, 2 = J1 max, 4 = J2 min, 8 = J2 max
9, 10	Referencia interna de q1, q2 en grados
11, 12	PWM de los motores 1 y 2

6. El panel de control (scara_teleop_ros.py)

Se abre con `python3 ros2/scara_teleop_ros.py` (necesita `sudo apt install python3-tk`). La línea superior muestra el estado en vivo, incluidos los finales de carrera presionados.

Teclado	Haz clic en la ventana, fuera de los cuadros de texto. **Mantén ESPACIO** (dead-man) y usa las flechas: izquierda/derecha mueven J1, abajo/arriba mueven J2. Al soltar el espacio el robot se detiene. Atajos: h Home, s Detener, o Apagar, c Borrar falla, Esc Detener, q Salir.
Acciones	Botones Home, Detener, Apagar motores, Borrar falla, Estado, Ayuda y Pulso J1 / J2 (pulso de prueba para verificar motor y encoder).
Mover en x, y, z	Escribe x, y, z en mm y pulsa Ir (o Enter). El selector de codo (+1 / -1) elige la configuración del brazo.
Mover en ángulos	Escribe q1 y q2 en grados y pulsa Ir.
Ganancias PID	kp, ki, kd, kff y fric por eje. Se llenan solas al conectar. Aplicar J1 / J2 las envía y relee lo que quedó. **Son temporales**: se pierden al reiniciar el ESP32; para fijarlas hay que editar config.c y reflashear.
Línea de comandos	Equivale al monitor serie: escribe help, status, gain, goto 120 80, etc. Flechas arriba/abajo recorren el historial.
Registro	Muestra los eventos del robot: verde = #LLEGO, rojo = #ERROR y #FALLA.
Seguridad	Si escribes en un cuadro de texto o la ventana pierde el foco, el teclado del robot se desactiva y el robot se detiene. Clic fuera de los cuadros (o Esc dentro de uno) devuelve el teclado al robot.

7. Errores frecuentes y cómo resolverlos

Síntoma / mensaje	Causa probable	Solución
No llega estado del ESP32. ¿Está corriendo el agente micro-ROS y el robot conectado?	El agente no está corriendo, el puerto o los baudios son distintos, hay un monitor serie abierto, o el ESP32 no se ha reiniciado.	Revisa la terminal 1. Cierra cualquier monitor serie. Confirma `-b 115200` y `/dev/ttyUSB0`. Pulsa EN en el ESP32.
docker: command not found / **permission denied**	Docker no instalado o tu usuario no está en el grupo docker.	`sudo apt install docker.io`, `sudo usermod -aG docker $USER` y reinicia la sesión. Mientras tanto usa `sudo docker run ...`.
/scara/scara_esp32 no aparece en `ros2 node list`	El agente no tiene sesión con el robot.	Reinicia el agente (Ctrl+C y vuelve a lanzarlo) y pulsa EN. Comprueba el cable USB.
#ERROR: primero ejecuta HOME	Se encendió el robot o se borró una falla: los ejes no tienen referencia.	Ejecuta `home`.
#FALLA: final de carrera inesperado (se perdio la referencia) al iniciar Home	Al empezar, el tope contrario al que busca el eje ya estaba presionado: el brazo quedó apoyado en un tope o un interruptor/cable da señal falsa.	**Apagar motores**, mueve el brazo a mano al centro del recorrido y mira en el panel la línea de finales de carrera: todo debe estar en '-'. Si un tope sigue en ON con el brazo libre, revisa el cable/interruptor. Luego **Borrar falla** y **Home**.
#FALLA: error de seguimiento o **motor trabado**	El motor no sigue la referencia: algo lo frena, el signo del motor/encoder está invertido, o la ganancia es baja.	Libera el brazo, **Borrar falla** y repite el Home. Si vuelve, prueba **Pulso J1/J2** para verificar el sentido y revisa las ganancias.
#FALLA: tiempo de Home agotado	No llegó a un tope a tiempo: el PWM de búsqueda es muy bajo o el brazo está trabado.	Revisa que nada estorbe. Si no se mueve, sube `HOME_DUTY` en config.h (valor actual 170) y reflashea.
#ERROR: no se puede ir a (x, y): inalcanzable...	La consigna está fuera del espacio de trabajo (50 a 190 mm de alcance) o es singular.	Elige un punto dentro de los círculos verdes de RViz o prueba el otro codo (+1 / -1).
#ERROR: no llego a la consigna a tiempo	El brazo se detuvo un poco antes de la tolerancia (fricción) y no cumplió el criterio de llegada.	Revisa las ganancias en el panel (sube fric o ki con cuidado) o repite el movimiento.
#ERROR: el robot esta ocupado (modo ...)	Hay otro movimiento o el Home en curso.	Pulsa **Detener** y vuelve a intentar.
El panel dice 'sin datos del ESP32'	No llega /scara/state.	Mismo caso que 'No llega estado': agente o ESP32.
El teleop no mueve el robot	Falta el Home, hay una falla, la ventana no tiene el foco, o no mantienes el espacio (dead-man).	Haz clic fuera de los cuadros de texto, mantén ESPACIO y mira que el indicador diga DEAD-MAN ON. Revisa Home y falla en la línea superior.
Se abre una terminal en vez de la ventana, o 'tkinter no está instalado'	Falta python3-tk.	`sudo apt install python3-tk`
RViz: icono rojo en SCARA / el robot no aparece	El nodo scara_rviz no está corriendo o no llega /scara/state.	`ros2 node list` debe mostrar /scara_rviz. Si no, ejecuta `python3 ros2/scara_rviz.py` para ver el error. Verifica que Fixed Frame sea base_link.
RViz: ejes de colores gigantes	Está activo el display TF.	Desmarca la casilla 'TF' en el panel izquierdo.
RViz: el robot de la pantalla gira al revés del real	El signo de un ángulo no coincide entre el firmware y el URDF.	Invierte el signo de ese ángulo en el URDF (o en scara_rviz.py).
AssertionError PyFloat_Check al ejecutar un script	Versión vieja de scara_rviz.py (enteros en campos de ROS).	Usa la versión actual del archivo.
`Could not exclusively lock port /dev/ttyUSB0`	Otro programa tiene el puerto.	`fuser -v /dev/ttyUSB0` y cierra el proceso (`kill PID`). Sirve también para el agente de Docker.

Síntoma / mensaje	Causa probable	Solución
El robot se detiene a ratos en modo manual	Se pierden latidos del dead-man (300 ms de tolerancia): PC lento o USB inestable.	Es el comportamiento seguro. Cierra programas pesados y revisa el cable USB.

Si hay que volver a compilar el firmware

Solo hace falta si cambias `config.c` o `config.h`. Compila siempre con un entorno limpio, sin ROS cargado:

```
env -i HOME="$HOME" TERM="$TERM" \
  PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:$HOME/.local/bin" \
  bash --noprofile --norc -c \
  'cd ~/Trabajos/Robotica/Robot/scara_fw_microros/scara_fw && pio run -t upload'
```

Error al compilar	Solución
`Unknown boolean field "use_gitignore"`	`sed -i '/use_gitignore/d' components/micro_ros_espidf_component/idf_component.yml`
`CMake 3.20 or higher is required. You are running version 3.16.4`	Enlazar el CMake del sistema: `ln -s /usr/bin/cmake ~/.platformio/packages/tool-cmake/bin/cmake` (después de mover el original a cmake.orig).
`Could not find ament_cmake` / errores raros de ROS	Compilaste con ROS cargado: usa el comando `env -i ...` de arriba.
`*.obj: No such file` o `install: No such file` en micro_ros	Quedaron carpetas a medias: borra `components/micro_ros_espidf_component/{micro_ros_dev,micro_ros_src,libmicroros.a,include}` y compila de nuevo.

8. Al terminar

Cierra con Ctrl+C en este orden: el panel o teleop, RViz (terminal 2) y por último el agente (terminal 1). Los motores quedan en HOLD; si cortas la alimentación de potencia se sueltan. Al volver a encender hay que repetir el Home.